

CS3002E Compiler Design

Monsoon 2025

Lecture #28

Processing Declarations

Types and Relative Addresses for Names

Department of Computer Science & Engineering
NIT Calicut

Declarations

Declaration in a C like language: *int x; float y;*

What are the attributes in the Symbol Table entries for the variables x and y ?

Declarations

int x; float y;

Attributes in the Symbol Table entries for *x* and *y*
Name, Type, **Relative Address**

Relative Address

Local declaration in $f()$: *int* x ; *float* y ;

Address relative to the base of the data area for $f()$
 x at relative address 0, y at address?

Relative Address

Local declaration in $f()$: *int* x ; *float* y ;

x at relative address 0

y at address $0 + \text{size}(\text{int})$

Relative Address

```
int x[5]; float y;
```

If the first element of x is at relative address 0, what is the relative address of y ?

Storage layout for local names

- The *width* of a type - number of storage units needed for objects of that type
- The amount of storage required at run time for a name can be determined based on its type
- Compiler assigns each name a relative address, stores type and relative address in the Symbol Table

Syntax-Directed Translation Scheme (SDT)

A notation for specifying a translation:

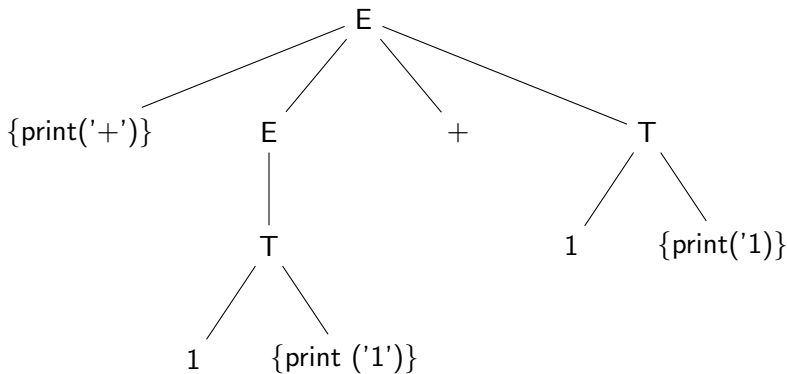
- a Context Free Grammar with *semantic actions* embedded within production bodies
- the order of evaluation of semantic rules is explicitly specified

$$E \rightarrow \{ \textit{print}(' + ') \} \ E + T$$
$$E \rightarrow T$$
$$T \rightarrow 1 \ \{ \textit{print}('1') \}$$

$E \rightarrow \{ \text{print}(' + ') \} E + T$

$E \rightarrow T$

$T \rightarrow 1 \{ \text{print}('1') \}$



Input: 1+1

Do a left-to-right depth-first traversal of the tree; when a leaf node for a semantic action is visited, execute the semantic action.

Postfix SDT - all actions at the right ends of the production bodies

$$E \rightarrow E + T \quad \{print('+')\}$$

$$E \rightarrow T$$

$$T \rightarrow 1 \quad \{print('1')\}$$

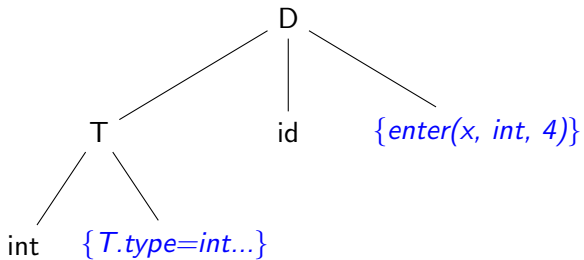
Input: 1+1

Computing Types and their Widths

$$D \rightarrow T \text{ id } \{ \text{enter}(\text{id.lexeme}, T.\text{type}, T.\text{width}) \}$$
$$T \rightarrow \text{int} \quad \{ T.\text{type} = \text{int}; T.\text{width} = 4 \}$$
$$T \rightarrow \text{float} \quad \{ T.\text{type} = \text{float}; T.\text{width} = 8 \}$$

Parse tree with semantic actions for `int x` ?

int x



Sequence of Declarations

$$P \rightarrow D$$

$$D \rightarrow D; D$$

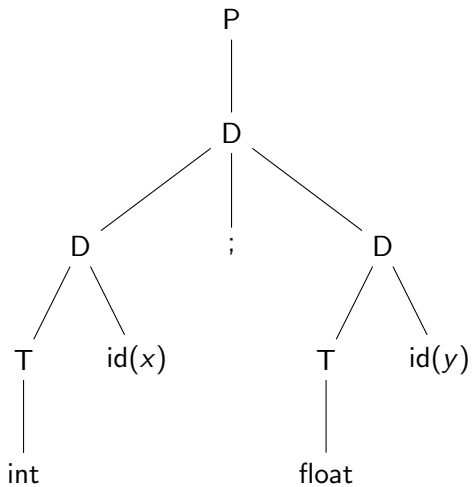
$$D \rightarrow T \textit{id}$$

$$T \rightarrow \textit{int}$$

$$T \rightarrow \textit{float}$$

Draw parse tree for
int x; float y

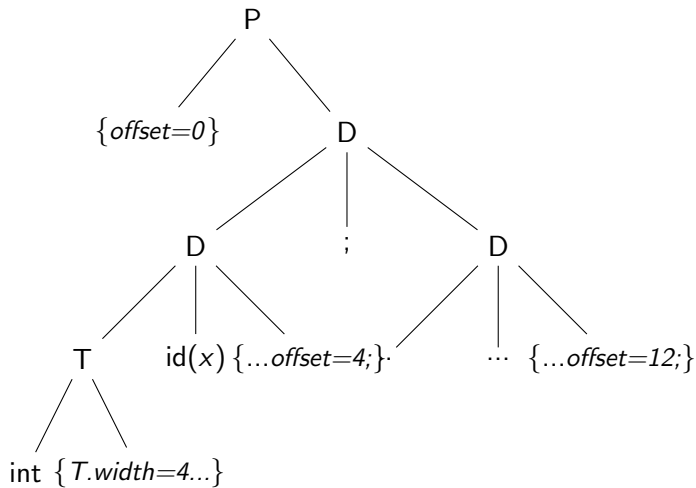
int x; float y



Assigning Relative Addresses to Names

$$P \rightarrow \{offset = 0\} D$$
$$D \rightarrow D; D$$
$$D \rightarrow T \text{ id } \{enter(id.lexeme, T.type, offset); \\ offset = offset + T.width)\}$$
$$T \rightarrow int \{T.type = int; T.width = 4\}$$
$$T \rightarrow float \{T.type = float; T.width = 8\}$$

int x; float y



Arrays

```
int [2] x;  
float [3] [2] y;
```

Type Expression?

Width?

Arrays

int [2] x;

Type Expression: *array*(2, *int*)

Width: 2×4

Arrays

```
float [3] [2] y;
```

Type Expression: *array*(3, *array*(2, *float*))

Width: $3 \times 2 \times 8$

Array Declaration: one-dimensional integer array

$$T \rightarrow \text{int } [\text{num}] \quad \{ T.type = \text{array}(\text{num.lexval}, \text{int}); \\ T.width = \text{num.lexval} \times 4 \}$$

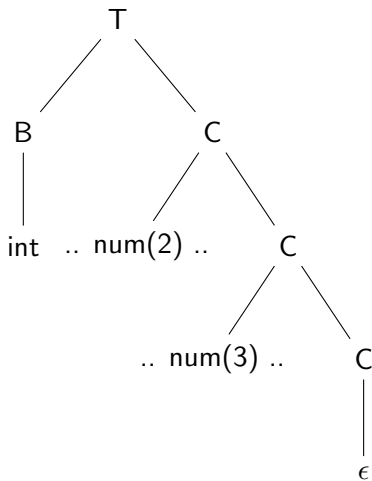
Parse Tree with semantic actions for *int* [3] ?

Array Declaration: general

$$T \rightarrow B C$$
$$B \rightarrow \textit{int} \mid \textit{float}$$
$$C \rightarrow [\textit{num}] C \mid \epsilon$$

Draw the parse tree for *int* [2] [3]

int [2] [3]



Array Declaration: computing type and width

$$T \rightarrow B \{t = B.type; w = B.width\} \quad C$$

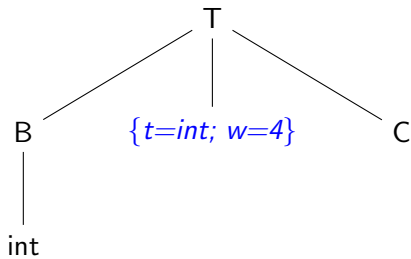
$$B \rightarrow int \{B.type = int; B.width = 4\}$$

$$B \rightarrow float \{B.type = float; B.width = 8\}$$

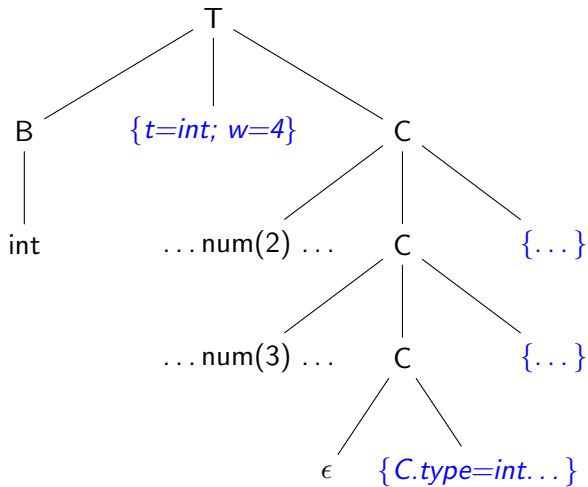
$$C \rightarrow [num] C_1 \{C.type = array(num.lexval, C_1.type) \\ C.width = num.lexval \times C_1.width\}$$

$$C \rightarrow \epsilon \{C.type = t; C.width = w\}$$

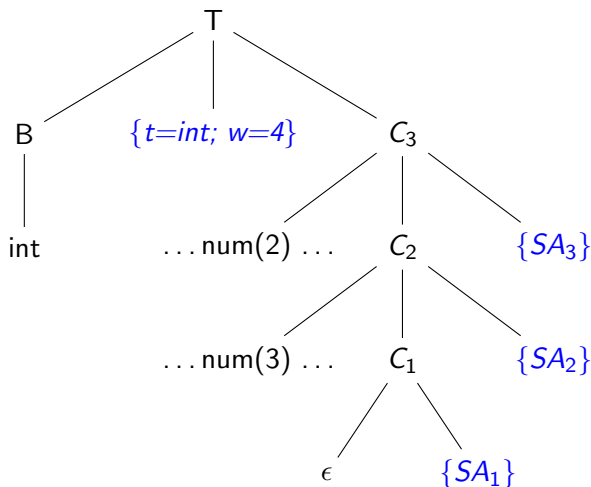
int [2] [3]



int [2] [3]



int [2] [3]



$SA_1 : \{ C_1.type = int; C_1.width = 4 \}$

$SA_2 : \{ C_2.type = array(3, int); C_2.width = 12 \}$

$SA_3 : \{ C_3.type = array(2, array(3, int)); C_3.width = 24 \}$

Array Declaration: computing type and width

$$T \rightarrow B \{t = B.type; w = B.width\} \quad C$$

$$B \rightarrow int \{B.type = int; B.width = 4\}$$

$$B \rightarrow float \{B.type = float; B.width = 8\}$$

$$C \rightarrow [num] C_1 \{C.type = array(num.lexval, C_1.type) \\ C.width = num.lexval \times C_1.width\}$$

$$C \rightarrow \epsilon \{C.type = t; C.width = w\}$$

Exercise: Include the semantic sections for setting $T.type$ and $T.width$.

References

References:

- Aho A.V., Lam M.S., Sethi R., and Ullman J.D. Compilers: Principles, Techniques, and Tools (ALSU). Pearson Education, 2007.

Further reading:

- ALSU Section 6.3